CHARITY AND AID MANAGEMENT SYSTEM

Design Documentation

Version 3.0 — Final

Prepared By:	Shivani Pothkanuri
Advisor:	Jeffrey Tirschman
Course:	AIT 725 — Case Study Implementation
Institution:	Towson University
Date:	March 21, 2026
Semester:	Spring 2026

Revision History

The table below tracks all versions of this Design Documentation. Each row records the version number, date, author, description of changes, and the document status at that revision.

Version	Date	Author	Description of Changes	Status
V1.0	02/16/2026	Shivani Pothkanuri	Initial design documentation draft submitted for advisor review.	Draft
V2.0	03/01/2026	Shivani Pothkanuri	Revised per advisor feedback: added TOC, fixed section numbering, expanded physical data model.	Draft
V3.0	03/21/2026	Shivani Pothkanuri	Final version: added UI mockup descriptions, updated tech stack to Java/Spring Boot/MySQL, corrected ERD note, expanded security and API sections.	Final

Table of Contents

1. System Overview

The Charity and Aid Management System (CAMS) is a three-tier web application designed to centralize the donation, campaign, beneficiary, and aid distribution operations of small and mid-sized charitable organizations. This design document defines the technical architecture, technology stack, security model, user interface design, physical data model, API specification, and sprint development plan that will guide the construction of the system. It translates the requirements established in the Software Requirements Specification into actionable design decisions and implementation guidance.

The system is built on a clearly separated architecture: an HTML5 and Bootstrap 5 presentation layer, a Java 17 and Spring Boot 3 application layer, and a MySQL 8 data layer accessed through Spring Data JPA with Hibernate. All authentication is stateless JWT-based, managed by Spring Security, with role-based access control enforced at the API level using @PreAuthorize annotations. The system is deployed to a cloud hosting platform (AWS EC2 or Railway) with GitHub Actions providing a CI/CD pipeline.

1.1 Purpose

The purpose of this Design Documentation is to provide a complete technical specification for the development of the Charity and Aid Management System. It serves as the primary reference for all implementation decisions, establishing how the system will be structured, how data will be stored and accessed, how users will be authenticated and authorized, and how the user interface will be organized. This document is the direct basis for sprint planning, development, testing, and deployment activities throughout the Spring 2026 semester.

1.2 Scope

This document covers the full technical design of all system components defined in the SRS, including the three-tier architecture, the complete technology stack, all six core database entities with their physical schemas, all 35 RESTful API endpoints, the five role-based UI screen designs, the security model, business rules, and the three-sprint development plan. Excluded from scope are payment gateway integration, native mobile applications, and third-party CRM integrations, which were identified as out of scope in the SRS.

1.3 System Context

The Charity and Aid Management System operates as a cloud-hosted web application accessed through modern browsers. Five user roles interact with the system: Donors submit and track donations; Staff manage campaigns, beneficiaries, and aid requests; Case Managers review and decide on aid requests; Administrators manage the full system; and Leadership monitors organizational KPIs. Externally, the system communicates with an SMTP email service (SendGrid) for automated notifications and with the MySQL 8 database for all persistent data operations. All traffic between the browser and the server travels over HTTPS using TLS 1.2 or higher.

1.4 Key Features

- Three-tier architecture: Bootstrap 5 frontend, Java 17 / Spring Boot 3 backend, MySQL 8 database.
- Spring Security JWT authentication with role-based access control (@PreAuthorize) for all five user roles.
- Full campaign lifecycle: creation, progress tracking, automatic completion on goal achievement.
- Donation recording with unique transaction IDs, PDF receipt generation, and automated email confirmation.
- Beneficiary registration and aid request workflow with case manager approval, denial, and escalation.
- Aid fulfillment with inventory deduction and low-stock alerting for in-kind items.
- Reporting and KPI dashboard for leadership with role-restricted access enforced by RBAC.
- Full audit trail using SLF4J/Logback for all create, update, approval, and login events.
- GitHub Actions CI/CD pipeline with automated build, test, and deployment to cloud hosting.

1.5 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
CAMS	Charity and Aid Management System — the web application being designed and built in this project.
SRS	Software Requirements Specification — the companion document defining all system requirements.
JWT	JSON Web Token — a signed, stateless token used for user authentication and session management.
RBAC	Role-Based Access Control — the security mechanism restricting access based on assigned user role.
ORM	Object-Relational Mapping — the technique mapping Java entity classes to database tables (Hibernate).
JPA	Java Persistence API — the standard Java specification for ORM, implemented by Hibernate.
API	Application Programming Interface — the RESTful endpoints exposed by the Spring Boot backend.
ERD	Entity Relationship Diagram — a conceptual model showing database entities and their relationships.
CI/CD	Continuous Integration / Continuous Deployment — automated build and deployment pipeline via GitHub Actions.
CRUD	Create, Read, Update, Delete — the four basic database operations applied to every system entity.
MFA	Multi-Factor Authentication — additional login verification required for Administrator accounts.
SMTP	Simple Mail Transfer Protocol — the email delivery protocol used by SendGrid for notifications.
TLS	Transport Layer Security — the encryption protocol used for all HTTPS communication.
WCAG	Web Content Accessibility Guidelines — accessibility standards the UI must satisfy (Level AA).
KPI	Key Performance Indicator — summary metrics displayed on the leadership dashboard.

2. References

The following references informed the design decisions and documentation standards applied in this document. All technology documentation was consulted to confirm version-specific capabilities and constraints relevant to the approved stack.

Reference	Source / URL
Spring Boot 3 Documentation	https://docs.spring.io/spring-boot/docs/current/reference/html/
Spring Security Reference	https://docs.spring.io/spring-security/reference/
Spring Data JPA / Hibernate	https://docs.spring.io/spring-data/jpa/docs/current/reference/html/
MySQL 8.0 Reference Manual	https://dev.mysql.com/doc/refman/8.0/en/
Bootstrap 5 Documentation	https://getbootstrap.com/docs/5.3/getting-started/introduction/
OWASP Top Ten 2021	https://owasp.org/Top10/ — Security best practices for web applications.
IEEE Std 1016-2009 (Software Design)	IEEE Standard for Information Technology — Systems Design — Software Design Descriptions.
RFC 7519 — JSON Web Tokens	https://www.rfc-editor.org/rfc/rfc7519 — JWT specification.
W3C WCAG 2.1	https://www.w3.org/TR/WCAG21/ — Web Content Accessibility Guidelines Level AA.
GitHub Actions Documentation	https://docs.github.com/en/actions — CI/CD pipeline configuration reference.

3. Coding Languages, Libraries, and Technologies

The Charity and Aid Management System is built on a three-tier architecture. Each tier uses a defined set of languages, frameworks, and libraries chosen to satisfy the functional and non-functional requirements specified in the SRS. The technology decisions below are final and binding for all development sprints. No technology substitutions are permitted without formal advisor approval and a revision of this document.

3.1 Frontend Development

The presentation layer is responsible for rendering the user interface and handling user interactions in the browser. It communicates with the backend exclusively through RESTful API calls over HTTPS, consuming JSON responses. The frontend has no direct database access.

3.1.1 Languages

Language	Purpose and Usage
HTML5	Structure of all web pages, forms, tables, and navigation components.
CSS3	Styling of all pages in conjunction with Bootstrap utility classes.
JavaScript (ES6+)	Dynamic UI behavior, form validation, API calls via Fetch API, DOM manipulation.

3.1.2 Libraries and Frameworks

Library / Framework	Purpose and Usage
Bootstrap 5.3	Primary UI framework for responsive grid layout, navigation, forms, buttons, modals, cards, and progress bars. Ensures WCAG 2.1 Level AA compliance through semantic HTML and accessible components.
Fetch API (native browser)	Used for all async HTTP calls to the Spring Boot REST API. Replaces third-party HTTP clients to minimize frontend dependencies.
Chart.js 4.x	Client-side chart rendering for the leadership KPI dashboard (line charts, bar charts, donut charts). Loaded via CDN.

3.2 Backend Development

The application layer implements all business logic, security enforcement, data validation, and API routing. It is the only tier that communicates with the MySQL 8 database, ensuring the data layer is never directly exposed to the browser or external clients.

3.2.1 Language

Java 17 (LTS) is the sole backend language. Java 17 was selected for its enterprise-grade stability, strong typing, broad Spring ecosystem support, and alignment with the AIT 725 course requirements.

3.2.2 Framework and Libraries

Library / Framework	Purpose and Usage
Spring Boot 3.2	Core application framework. Provides auto-configuration, embedded Tomcat server, and production-ready defaults. Enables rapid development of the three-tier architecture.
Spring MVC (@RestController)	Handles all HTTP request routing, JSON serialization/deserialization, and RESTful API endpoint definition.
Spring Security 6	Provides JWT-based stateless authentication, role-based access control via @PreAuthorize annotations, CSRF protection, session management, and account lockout enforcement.

Library / Framework	Purpose and Usage
Spring Data JPA + Hibernate 6	ORM layer for all database operations. Hibernate maps Java entity classes to MySQL tables. Spring Data JPA provides repository interfaces for CRUD and custom queries.
JJWT (Java JWT Library)	Used for generating, signing, and validating JWT tokens for stateless user session management.
JavaMail / SendGrid Java SDK	Backend email delivery for donation confirmations, aid decision notifications, and campaign alerts.
iText 7 / Apache PDFBox	PDF generation for downloadable donation receipts.
SLF4J + Logback	Structured server-side logging for application events and the system audit trail.
Spring Boot Actuator	Health checks and operational metrics endpoints for monitoring the deployed application.

3.3 Database Management

The data layer uses MySQL 8 as the relational database management system. MySQL 8 was selected for its proven reliability, strong support for ACID-compliant transactions, and compatibility with Spring Data JPA and Hibernate. All schema creation and management is handled through Hibernate entity mappings, with DDL auto-generation enabled in the development environment and migration scripts used for production deployment.

Component	Detail
RDBMS	MySQL 8.0
ORM	Spring Data JPA with Hibernate 6 — all entity classes annotated with @Entity, @Table, @Column.
Connection Pooling	HikariCP (bundled with Spring Boot) — manages the database connection pool.
Transactions	Spring @Transactional annotations enforce atomicity for all multi-step operations (e.g., donation recording + campaign total update).
Referential Integrity	Primary and foreign key constraints enforced at the MySQL level, not just ORM level.
Hosting	MySQL instance hosted on the same cloud platform as the Spring Boot application (Railway or AWS RDS).

3.4 Authentication and Security

Security is implemented at multiple layers of the architecture. Spring Security is the sole authentication and authorization framework. Custom or third-party authentication libraries are not permitted per project constraints.

Mechanism	Implementation Detail
JWT Authentication	On successful login, the server issues a signed JWT containing the user's ID, role, and expiry (15-minute inactivity). The client includes the token in the Authorization: Bearer header of every API request.
Password Hashing	BCryptPasswordEncoder with cost factor 12. Applied at registration and password reset. Plain-text passwords never touch the database.
RBAC Enforcement	Spring Security @PreAuthorize annotations on all @RestController methods. Each endpoint specifies the minimum required role(s).

Mechanism	Implementation Detail
MFA for Administrators	TOTP-based multi-factor authentication enforced for all users with the Administrator role at login.
Account Lockout	After 5 consecutive failed login attempts, the account status changes to Suspended and an admin notification is generated.
CSRF Protection	Spring Security CSRF tokens enabled for all state-changing HTTP methods (POST, PUT, DELETE).
HTTPS / TLS	All traffic encrypted using TLS 1.2+. HTTP-only access is blocked at the server configuration level.
Input Validation	Server-side validation using Jakarta Bean Validation (@NotNull, @Size, @Email) on all request DTOs. Spring Data JPA parameterized queries prevent SQL injection.
XSS Prevention	Output encoding applied to all user-supplied data rendered in HTML responses.

3.5 Notifications

Automated email notifications are delivered through an SMTP integration using SendGrid. All notification logic is handled in the Spring Boot service layer and triggered by system events. No notification content is generated or stored in the frontend.

Notification Event	Trigger and Recipient
Donation confirmation	Triggered immediately after a donation is recorded. Sent to the Donor with transaction ID and PDF receipt attached.
Aid request approved	Triggered when a Case Manager approves a request. Sent to the beneficiary's email (or staff on their behalf).
Aid request denied	Triggered when a Case Manager denies a request. Sent with the documented justification note.
Campaign nearing goal	Triggered when collected amount reaches 90% of goal. Sent to authorized Staff and Administrators.
Low stock alert	Triggered when an inventory item falls below its defined low threshold. Sent to Staff.
High-value donation	Triggered when a single donation exceeds the configured threshold. Sent to Leadership.
Pending approval reminder	Triggered after a request has been Pending for more than 48 hours. Sent to Case Manager.
Password reset link	Triggered on user request. Link expires after 30 minutes. Sent to the registered email address.

3.6 Version Control and CI/CD

All source code is maintained in a GitHub repository using Git for version control. GitHub Actions automates the build, test, and deployment pipeline, ensuring that every push to the main branch is validated before deployment.

Component	Detail
Version Control	Git — all commits follow a feature-branch workflow. Main branch is protected and requires passing CI checks before merge.
Repository	GitHub — single repository containing backend (src/main/java), frontend (src/main/resources/static), and database migration scripts.
CI Pipeline (GitHub Actions)	On every push: (1) Build Spring Boot application with Maven; (2) Run unit and integration tests; (3) Check code style with Checkstyle.
CD Pipeline (GitHub Actions)	On merge to main: (1) Build production JAR; (2) Deploy to Railway or AWS EC2 via SSH; (3) Run smoke tests against deployed URL.

3.7 Deployment and Hosting

The system will be deployed to a cloud hosting platform to ensure public accessibility via HTTPS for advisor review and demonstration. Railway is the primary deployment target due to its native Spring Boot support and integrated MySQL plugin. AWS EC2 is the secondary option if Railway limitations are encountered.

Component	Detail
Primary Hosting	Railway.app — supports Java Spring Boot applications via Nixpacks build. MySQL plugin provides a managed database instance on the same platform.
Secondary Hosting	AWS EC2 (t2.micro, Free Tier) — Spring Boot JAR deployed as a systemd service. AWS RDS (MySQL 8) for the database.
Domain / URL	Public HTTPS URL generated by Railway or configured on EC2 (e.g., https://cams-app.up.railway.app).
Environment Variables	Database URL, JWT secret, SendGrid API key, and SMTP credentials stored as environment variables — never hardcoded in source code.
SSL/TLS	Managed automatically by Railway. On EC2, configured via Nginx reverse proxy with Let's Encrypt certificate.

4. Business Rules

Business rules define the specific operational constraints and logic that govern how the system processes data, enforces policies, and manages workflows. Each rule is implemented as server-side logic within the Spring Boot application layer and cannot be bypassed by any user role or client-side action. These rules are derived from the functional requirements in the SRS and represent binding constraints on system behavior throughout all development sprints.

Rule ID	Business Rule Description
BR-01	All users must register an account and provide a valid name, email address, and password before accessing any system feature.
BR-02	System access and functionality shall be restricted based on the user's assigned role. Valid roles are: Donor, Staff, Case Manager, Administrator, and Leadership. Each role has specific permissions enforced by Spring Security @PreAuthorize annotations.
BR-03	Only authorized Organizational Staff or Administrators may create, update, or close fundraising campaigns.
BR-04	Donors may submit monetary or in-kind donations only to campaigns with a status of Active. Donations to Completed or Closed campaigns are rejected by the system.
BR-05	Each donation transaction must be recorded with a unique system-generated Transaction ID, the donation amount or item description, the donor's user ID, and the linked Campaign ID.
BR-06	A beneficiary must be registered by authorized Staff before any aid request can be submitted or recorded on their behalf.
BR-07	Aid requests must include the beneficiary ID, aid type, a description of the assistance needed, and the requested amount or item before they can be saved and queued for review.
BR-08	A duplicate aid request for the same beneficiary and aid type may not be submitted within a 30-day window. The system will block the submission and notify the staff member.
BR-09	Aid requests must be reviewed and either approved or denied by a Case Manager or Administrator before any fulfillment action may occur. A justification note is required when denying a request.
BR-10	Aid fulfillment may only be recorded after the aid request status is Approved. The fulfillment date and the staff member who recorded it must be captured.
BR-11	For in-kind aid fulfillment, the system must verify that sufficient inventory exists before permitting fulfillment. If inventory is insufficient, fulfillment is blocked and staff are alerted.
BR-12	All mandatory fields in forms for donations, campaigns, aid requests, and beneficiaries must be completed before the record can be saved. The system shall enforce server-side validation for all required fields.
BR-13	Donation and financial transaction records may not be deleted by any user role. Deactivation is the only permitted removal action for user accounts.
BR-14	The system shall generate summary reports for donations, campaign performance, and aid distribution. Report access is restricted to authorized roles via RBAC.
BR-15	User account passwords must be stored as bcrypt hashes with a minimum cost factor of 12. Plain-text passwords must never be persisted in any system component.

5. Security Tiers and Rules

The Charity and Aid Management System implements a layered security model using Spring Security with JWT-based stateless authentication and Role-Based Access Control (RBAC). Every API endpoint is protected by @PreAuthorize annotations that verify the authenticated user's role before processing any request. Security is enforced exclusively on the server side — no security-sensitive logic exists in the frontend. This section defines the five user role security tiers, data security rules, authentication and authorization mechanisms, and network security controls.

5.1 User Role-Based Security Tiers

The following table defines each user role, its access level classification, and the full set of permitted system actions. All permissions are enforced by Spring Security at the API layer. Role assignments are managed exclusively by Administrators and are stored in the database as the USER_ROLE field on the tbl_users table.

Role	Access Level	Permitted Actions
Donor	Standard	Register account; log in; browse active campaigns; submit monetary or in-kind donations; download PDF receipts; view personal donation history; schedule/cancel recurring donations; update own profile.
Organizational Staff	Elevated	All Donor permissions; create/update/close campaigns; register/update beneficiaries; record aid requests; record fulfillment of approved requests; manage inventory items; view campaign and donation reports.
Case Manager	Elevated	View all pending aid requests; review full request details and beneficiary aid history; approve or deny aid requests with mandatory justification notes; escalate requests to administrator; monitor fulfillment status.
Administrator	Full	All Staff and Case Manager permissions; register and manage all user accounts; assign and modify user roles; suspend/deactivate/reactivate accounts; unlock locked accounts; configure notification templates; view full audit log; generate all reports including compliance audit report; enforce MFA for admin accounts.
Leadership	Read-Only Executive	View-only access to KPI dashboard, campaign performance reports, aid distribution summaries, donation summary reports, and high-value donation notifications. No write access to any system data.

5.2 Data Security Rules

The following data security rules are applied at the application layer and database layer to ensure the integrity, confidentiality, and availability of all system data. These rules are non-negotiable and must be verified during system testing.

Security Rule	Implementation
Password storage	All passwords hashed with BCrypt (cost factor 12) via Spring Security BCryptPasswordEncoder. Plain text never persisted or logged.
Token security	JWT tokens signed with HS256 using a 256-bit secret stored as an environment variable. Tokens expire after 15 minutes of inactivity.
Data transmission	All HTTP traffic encrypted over TLS 1.2+. HTTP-only connections rejected at server level.

Security Rule	Implementation
Input validation	All incoming request bodies validated server-side using Jakarta Bean Validation. Invalid input returns HTTP 400 with error details.
SQL injection prevention	All database queries use Spring Data JPA repository methods or @Query with named parameters. Raw string concatenation in queries is prohibited.
Beneficiary data restriction	tbl_beneficiaries records accessible only to Staff, Case Managers, and Administrators. Donors and Leadership are blocked by @PreAuthorize at the API level.
Financial record protection	DELETE operations on tbl_donations and tbl_aid_requests are blocked at the service layer. Deactivation-only policy enforced for users.
Audit logging	SLF4J/Logback records all create, update, approve, deny, login, and logout events with actor USER_ID, action type, affected entity, timestamp, and IP address.
Log retention	Audit logs retained for a minimum of 12 months on the hosting platform.

5.3 User Authentication and Authorization

Authentication is stateless and JWT-based. On successful login, the Spring Boot server issues a signed JWT containing the user's ID, assigned role, and expiry timestamp. The client stores this token and includes it in the Authorization header of every subsequent API request. The server validates the token signature and expiry on every request before executing any business logic. Authorization is enforced at the method level using Spring Security's @PreAuthorize annotations, ensuring that even a valid token cannot grant access to functions outside the user's role.

Auth Mechanism	Detail
Login flow	POST /api/auth/login → Spring Security validates credentials → BCrypt password check → JWT issued → returned to client.
Token validation	Every API request → Spring Security filter intercepts → JWT extracted from Bearer header → signature verified → role loaded → @PreAuthorize checked.
Session timeout	JWT claims include issued-at time. Server rejects tokens where (current time – last activity) > 15 minutes. Client must re-authenticate.
MFA (Administrators)	After credential validation, Admin accounts receive a TOTP challenge. Login completes only after the correct 6-digit code is submitted.
Password reset	User requests reset → system generates a time-limited token (30 min) → email sent → user submits new password → BCrypt re-hash → token invalidated.
Account lockout	FAILED_LOGIN_COUNT increments on each failure. At count = 5, ACCOUNT_STATUS set to Suspended. Admin must manually reactivate or system auto-unlocks after 30 minutes.

5.4 Network Security

Network-level security protections are implemented at both the application layer and the hosting infrastructure layer to protect the system against external threats, unauthorized access, and data interception.

Control	Implementation
HTTPS enforcement	All client-server communication uses HTTPS with TLS 1.2+. Nginx reverse proxy (on EC2) or Railway platform terminates SSL and forwards to the Spring Boot application on port 8080.
CORS policy	Spring Security CORS configuration restricts API access to the deployed frontend origin. Cross-origin requests from unknown origins are rejected with HTTP 403.
Firewall	Cloud hosting firewall configured to allow only ports 80 (redirected to 443) and 443. SSH access on port 22 restricted to developer IP only.
Intrusion detection	Railway platform and AWS CloudWatch (on EC2) monitor for unusual traffic patterns, repeated auth failures, and unexpected resource consumption.
Rate limiting	Spring Boot rate-limiting filter applied to /api/auth/login to prevent brute-force attacks (max 10 requests per minute per IP).
Dependency scanning	GitHub Actions pipeline includes OWASP Dependency-Check to flag known vulnerabilities in third-party libraries on every build.

6. User Interface Design

The user interface of the Charity and Aid Management System is built using HTML5, CSS3, JavaScript, and Bootstrap 5.3. The design follows a role-driven layout model: each user role sees a tailored interface that presents only the screens, navigation items, and action controls relevant to their permissions. A consistent top navigation bar, left sidebar, and main content area are used across all role dashboards to ensure familiarity and reduce learning time. The UI is fully responsive and adapts to desktop, tablet, and mobile viewports using Bootstrap's grid system and utility classes.

6.1 UI Design Principles

The following principles govern all interface design decisions and must be applied consistently across all screens and components throughout development.

Principle	Application
Simplicity and clarity	Each screen presents one primary task. Complex workflows are broken into clearly labeled steps. Jargon is avoided in all labels, buttons, and messages.
Consistency	The same navigation structure, color scheme, button styles, typography, and table formats are used across all screens and all roles.
Feedback and visibility	Every user action produces visible feedback: loading indicators, success banners, error messages, and confirmation dialogs for all destructive or critical actions.
Role-appropriate views	Navigation items and action buttons are rendered conditionally based on the authenticated user's role. Users never see controls they are not permitted to use.
Accessibility (WCAG 2.1 AA)	All form fields have associated labels, all images have alt text, color contrast ratios meet AA standards, and the interface is fully navigable by keyboard.
Responsive design	Bootstrap 5 grid and responsive utility classes ensure all screens render correctly on desktop (1280px+), tablet (768–1279px), and mobile (< 768px).

6.2 Layout and Navigation Structure

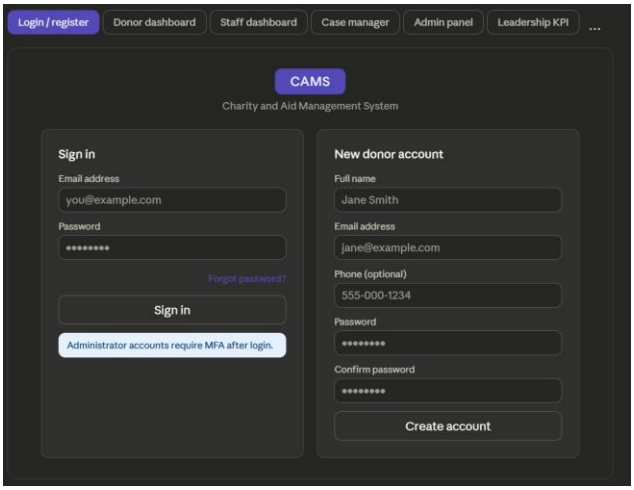
All authenticated role dashboards share the same three-region layout structure: a fixed top navigation bar, a fixed left sidebar, and a scrollable main content area. The top navigation bar contains the CAMS branding logo on the left, a global search input in the center, and a notifications bell icon and user profile dropdown on the right. The left sidebar lists the primary navigation modules available to the authenticated role, with a count badge on items requiring attention (e.g., Pending Requests: 7). The main content area loads the selected module view without a full page reload using JavaScript fetch and DOM update.

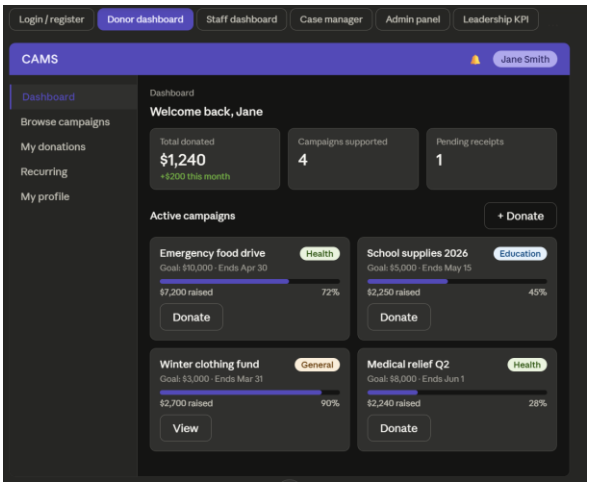
Navigation Element	Detail
Top navigation bar	Fixed at the top of all authenticated pages. Contains: CAMS logo (left), global search bar (center), notification bell with unread count badge (right), user avatar and name with dropdown (right). Dropdown options: My Profile, Settings, Logout.
Left sidebar	Fixed on the left. Collapses to icon-only mode on tablet/mobile. Navigation items are role-filtered. Active item is highlighted. Count badges on Pending Requests (Case Manager) and Low Stock Alerts (Staff).
Main content area	Scrollable. Displays the active module. Consistent inner structure: page title bar (with breadcrumb and primary action button), optional KPI card row, and primary data table or form.
Breadcrumb navigation	Displayed below page title on all secondary pages. Shows path from Dashboard to current screen. Each segment is a clickable link.

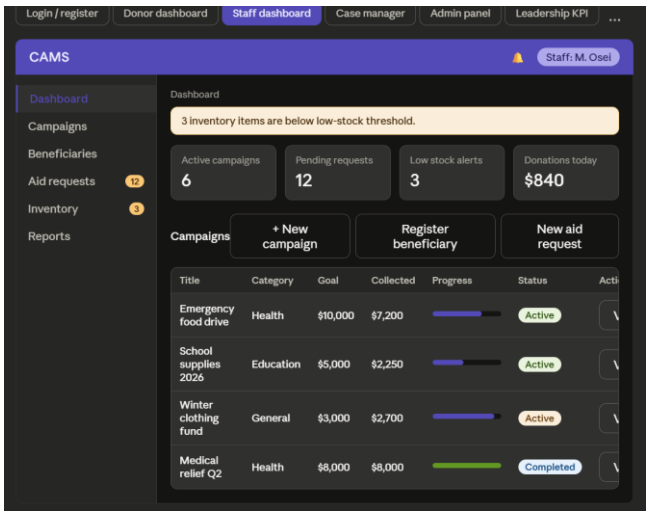
Navigation Element	Detail
Data tables	All list views use Bootstrap table-striped, table-hover classes with: sortable column headers, search/filter row above the table, pagination controls below, and row-level action buttons (View, Edit, Status Change).

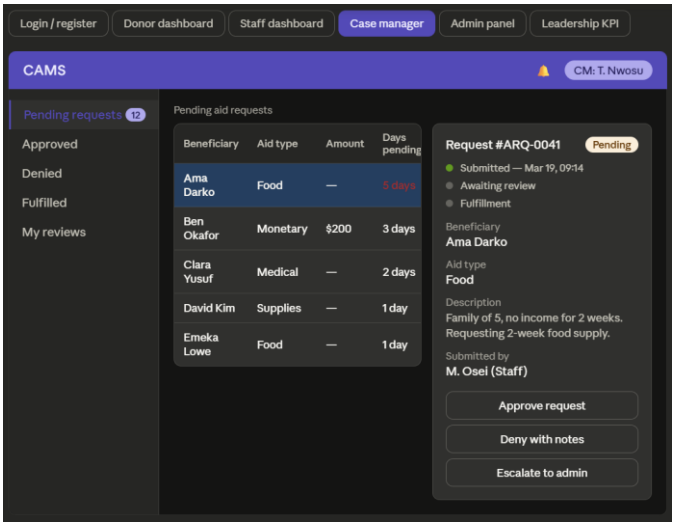
6.3 Screen Designs by Role

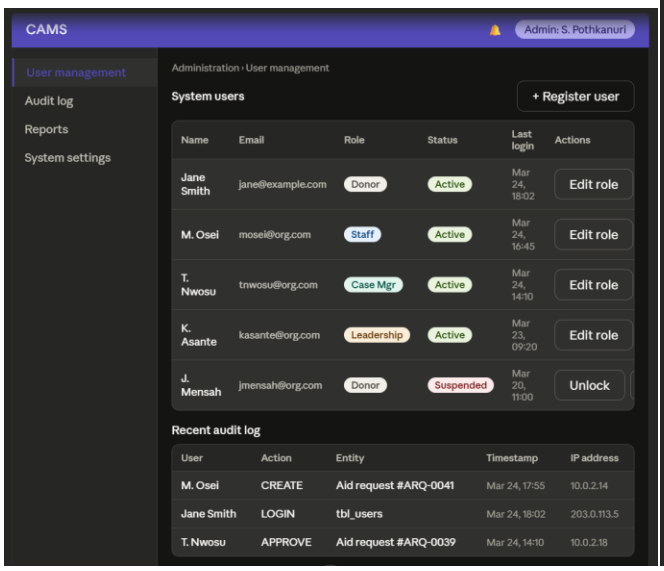
The following table describes the detailed UI layout, components, and controls for each of the six primary screens — one per user role plus a shared login screen. These descriptions serve as the specification for frontend development and must be implemented as described before the Sprint 2 peer review. Actual wireframe images are attached as Appendix A of this document.

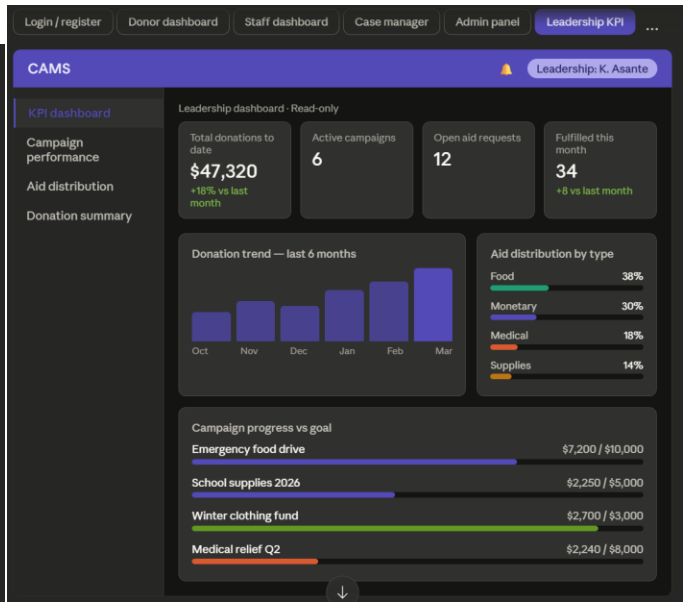












Screen	Role	Layout, Components, and Controls
Login / Registration Screen	All Users	Top navigation bar with CAMS branding logo and tagline. Two-column centered card layout: left panel with login form (Email, Password, Login button, Forgot Password link); right panel with New User registration form (Name, Email, Phone, Password, Confirm Password, Register button). Bootstrap form validation with inline error messages. MFA prompt appears post-login for Administrator accounts.
Donor Dashboard	Donor	Top navigation bar (logo, Donate Now CTA button, notification bell, user profile dropdown). Left sidebar navigation: Dashboard, Browse Campaigns, My Donations, Recurring Donations, My Profile. Main area: welcome banner with donor name; row of 3 KPI cards (Total Donated, Campaigns Supported, Pending Receipts); paginated Campaign Cards grid (card per active campaign showing title, category badge, progress bar, goal vs collected, Donate button). Quick Donate modal triggered from campaign card.
Staff Management Dashboard	Staff	Top navigation bar and sidebar (Dashboard, Campaigns, Beneficiaries, Aid Requests, Inventory, Reports). Main area: summary KPI row (Active Campaigns, Pending Aid Requests, Low Stock Alerts, Recent Donations); tabbed data table area with Campaign List (columns: Title, Category, Goal, Collected, Progress %, Status, Actions) and quick-access action buttons: New Campaign, Register Beneficiary, New Aid Request.
Case Manager Review Interface	Case Manager	Sidebar with pending request count badge. Main area split: left panel lists pending aid requests (sortable table: Beneficiary Name, Aid Type, Requested Amount, Submitted Date, Days Pending); right panel detail view shows full request details, beneficiary aid history accordion, supporting documentation section, Approve button (green), Deny button (red with mandatory notes modal), Escalate button. Status timeline shown at top of detail panel.
Administrator Control Panel	Administrator	Full-width dashboard. Four tabbed sections: (1) User Management — searchable user table with role badge, status indicator, and action buttons (Edit Role, Suspend, Deactivate, Unlock); (2) Audit Log — filterable

Screen	Role	Layout, Components, and Controls
		chronological log table (User, Action, Entity, Timestamp, IP Address); (3) Reports — report generation panel with type selector, date range picker, export buttons (PDF, CSV); (4) System Settings — notification template editor and high-value threshold configuration.
Leadership KPI Dashboard	Leadership	Read-only executive summary view. Top row: 4 large KPI metric cards (Total Donations to Date, Active Campaigns, Open Aid Requests, Aid Fulfilled This Month). Second row: Donation Trend chart (line chart, last 12 months) and Campaign Progress bar chart (goal vs collected per campaign). Third row: Aid Distribution donut chart (by type) and Resource Availability table. All data is real-time via API. No write controls visible for Leadership role.

6.4 Key UI Components

The following reusable components are implemented once and used consistently across all screens. Component styles are defined in a shared Bootstrap override CSS file (cams.css) loaded on every page.

Component	Description and Usage
KPI Metric Card	White card with subtle border shadow. Contains: muted 12px label above, bold 28px number below, optional colored delta indicator (up/down arrow with %). Used in rows of 3–4 on all dashboard screens.
Campaign Progress Card	Bootstrap card with campaign title, category badge (color-coded by category), Bootstrap progress bar (collected/goal %), goal and collected amounts, and a Donate / View button. Used in the Donor Campaign Browse grid.
Status Badge	Bootstrap badge component. Color-coded by status value: Active/Approved = green, Pending = amber, Denied = red, Completed/Fulfilled = blue, Inactive = gray. Used in all data tables and detail views.
Action Button Row	Standardized row of small Bootstrap outline buttons (View, Edit, Approve, Deny, Fulfill) at the end of each table row. Buttons visible only for roles with the corresponding permission.
Confirmation Modal	Bootstrap modal triggered before all destructive or irreversible actions (Delete, Deny, Deactivate). Contains action description, warning text, Cancel and Confirm buttons.
Notification Toast	Bootstrap toast positioned top-right. Appears on success (green), error (red), or warning (amber) events. Auto-dismisses after 5 seconds.
Side Panel Detail View	Used in Case Manager and Staff screens. On row click, a right-side panel slides in (Bootstrap offcanvas) showing full record details without navigating away from the list.

6.5 Responsive Design

The Bootstrap 5 grid system (12-column flexbox) ensures that all screens adapt automatically to the viewport width. On mobile devices (< 768px), the left sidebar collapses to a hamburger menu, KPI card rows stack vertically to a single column, data tables become horizontally scrollable within a fixed container, and modal dialogs expand to near full-screen. All form inputs, buttons, and touch targets meet the WCAG 2.1 minimum size of 44x44px on mobile viewports.

7. Physical Data Model

The Physical Data Model defines the exact database structure that will be implemented in MySQL 8 for the Charity and Aid Management System. Each table corresponds to a core system entity, with all columns specified using database-format field names (ALL_CAPS_NO_SPACES), precise MySQL data types with widths, a required/optional flag, valid dropdown values where applicable, and a plain-language description. Primary keys (PK) are auto-incremented integers. Foreign keys (FK) enforce referential integrity at the MySQL level in addition to the Hibernate ORM layer. All tables are managed through Spring Data JPA entity mappings with Hibernate as the ORM implementation.

Note: The Entity Relationship Diagram (ERD) showing all tables, primary keys, foreign keys, and cardinality relationships is located in the companion Software Requirements Specification document (Section 3), as directed by the academic advisor. The physical table definitions below serve as the implementation specification for each individual entity.

7.1 tbl_users

Stores all system user accounts across all roles. The USER_ROLE field determines the complete set of permissions granted to the account via Spring Security RBAC.

ID	Field Name (DB)	Data Type	Width	Required	Valid Values / Default	Description
PK	**USER_ID**	INT	—	Yes	Auto-increment	Unique system-generated identifier for each user account.
	FULL_NAME	VARCHAR	100	Yes	Free text	Full legal name of the registered user.
	EMAIL_ADDRESS	VARCHAR	150	Yes	Unique; used for login	Primary email. Must be unique across all accounts. Used for login and notifications.
	PASSWORD_HASH	VARCHAR	255	Yes	bcrypt hash only	Bcrypt-hashed password. Plain text never stored.
	PHONE_NUMBER	VARCHAR	20	No	Free text	Optional contact phone number.
	USER_ROLE	VARCHAR	30	Yes	Dropdown: Donor | Staff | Case Manager | Administrator | Leadership	Assigned system role. Controls all feature access via Spring Security RBAC.
	ACCOUNT_STATUS	VARCHAR	20	Yes	Dropdown: Active | Inactive | Suspended. Default: Active	Current lifecycle status of the account.
	CREATED_DATE	DATETIME	—	Yes	System-generated	Timestamp of account creation. Read-only.

ID	Field Name (DB)	Data Type	Width	Required	Valid Values / Default	Description
	LAST_LOGIN	DATETIME	—	No	System-updated	Timestamp of most recent successful authentication.
	FAILED_LOGIN_COUNT	INT	—	Yes	Default: 0. Resets on success	Count of consecutive failed login attempts. Account locks at 5.

7.2 tbl_campaigns

Stores all fundraising campaigns created by Staff and Administrators. COLLECTED_AMOUNT is maintained by the system and updated atomically within a @Transactional operation whenever a new donation is recorded.

ID	Field Name (DB)	Data Type	Width	Required	Valid Values / Default	Description
PK	**CAMPAIGN_ID**	INT	—	Yes	Auto-increment	Unique identifier for each fundraising campaign.
	CAMPAIGN_TITLE	VARCHAR	150	Yes	Free text	Descriptive title of the campaign shown to donors.
	DESCRIPTION	TEXT	2000	Yes	Free text	Detailed narrative description of the campaign purpose.
	GOAL_AMOUNT	DECIMAL	10,2	Yes	Must be > 0	Target monetary amount to be raised in USD.
	COLLECTED_AMOUNT	DECIMAL	10,2	Yes	System-maintained; default 0.00	Running total of all donations received. Updated automatically.
	START_DATE	DATE	—	Yes	Must be <= END_DATE	Date the campaign becomes active and accepts donations.

ID	Field Name (DB)	Data Type	Width	Required	Valid Values / Default	Description
	END_DATE	DATE	—	Yes	Must be >= START_DATE	Date the campaign closes to new donations.
	CAMPAIGN_STATUS	VARCHAR	20	Yes	Dropdown: Active | Completed | Closed. Auto-set when goal met	Current lifecycle status of the campaign.
	CATEGORY	VARCHAR	50	No	Dropdown: Health | Education | Emergency | Food | General	Classification used for filtering and reporting.
FK	**CREATED_BY**	INT	—	Yes	References tbl_users.USER_ID	User ID of the staff or admin who created the campaign.

7.3 tbl_donations

Stores every donation transaction. Records are immutable after creation — no update or delete operations are permitted on this table. RECEIPT_GENERATED is updated to true once the PDF receipt has been generated and the email sent.

ID	Field Name (DB)	Data Type	Width	Required	Valid Values / Default	Description
PK	**DONATION_ID**	INT	—	Yes	Auto-increment	Unique identifier for each donation transaction record.
	TRANSACTION_ID	VARCHAR	50	Yes	System-generated UUID	Unique human-readable reference code for the transaction.
FK	**DONOR_ID**	INT	—	Yes	References tbl_users.USER_ID	User ID of the donating user.
FK	**CAMPAIGN_ID**	INT	—	Yes	References tbl_campaigns.CAMPAIGN_ID	Campaign this donation is credited to.
	DONATION_TYPE	VARCHAR	20	Yes	Dropdown: Monetary | In-Kind	Type of contribution

ID	Field Name (DB)	Data Type	Width	Required	Valid Values / Default	Description
						being recorded.
	DONATION_AMOUNT	DECIMAL	10,2	Cond.	Required if DONATION_TYPE = Monetary; must be > 0	Monetary value of the donation in USD.
	ITEM_DESCRIPTION	VARCHAR	255	Cond.	Required if DONATION_TYPE = In-Kind	Description of the in-kind goods donated.
	PAYMENT_METHOD	VARCHAR	30	Cond.	Dropdown: Credit Card | Debit Card | Bank Transfer | N/A	Required for Monetary donations.
	DONATION_DATE	DATETIME	—	Yes	System-generated	Timestamp when the donation was recorded.
	RECEIPT_GENERATED	BOOLEAN	—	Yes	Default: false	Flag indicating whether the PDF receipt has been generated.

7.4 tbl_beneficiaries

Stores all registered beneficiary profiles. Duplicate detection is enforced at the service layer by checking the combination of FULL_NAME and PHONE_NUMBER before creating a new record.

ID	Field Name (DB)	Data Type	Width	Required	Valid Values / Default	Description
PK	**BENEFICIARY_ID**	INT	—	Yes	Auto-increment	Unique identifier for each registered beneficiary.
	FULL_NAME	VARCHAR	100	Yes	Free text	Full legal name of the beneficiary.
	EMAIL_ADDRESS	VARCHAR	150	No	Unique if provided	Contact email for notifications, if available.
	PHONE_NUMBER	VARCHAR	20	Yes	Free text	Primary contact phone number. Used

ID	Field Name (DB)	Data Type	Width	Required	Valid Values / Default	Description
						for duplicate check with FULL_NAME.
	RESIDENTIAL_ADDRESS	VARCHAR	255	Yes	Free text	Physical residential address.
	BENEFICIARY_STATUS	VARCHAR	20	Yes	Dropdown: Active | Inactive. Default: Active	Current lifecycle status of the beneficiary record.
FK	**REGISTERED_BY**	INT	—	Yes	References tbl_users.USER_ID	User ID of the staff member who registered this beneficiary.
	REGISTRATION_DATE	DATETIME	—	Yes	System-generated	Timestamp of initial beneficiary registration.

7.5 tbl_aid_requests

Stores the complete lifecycle of every aid request from submission through fulfillment. All FK fields referencing USER_ID (SUBMITTED_BY, REVIEWED_BY, FULFILLED_BY) capture accountability at each stage of the workflow.

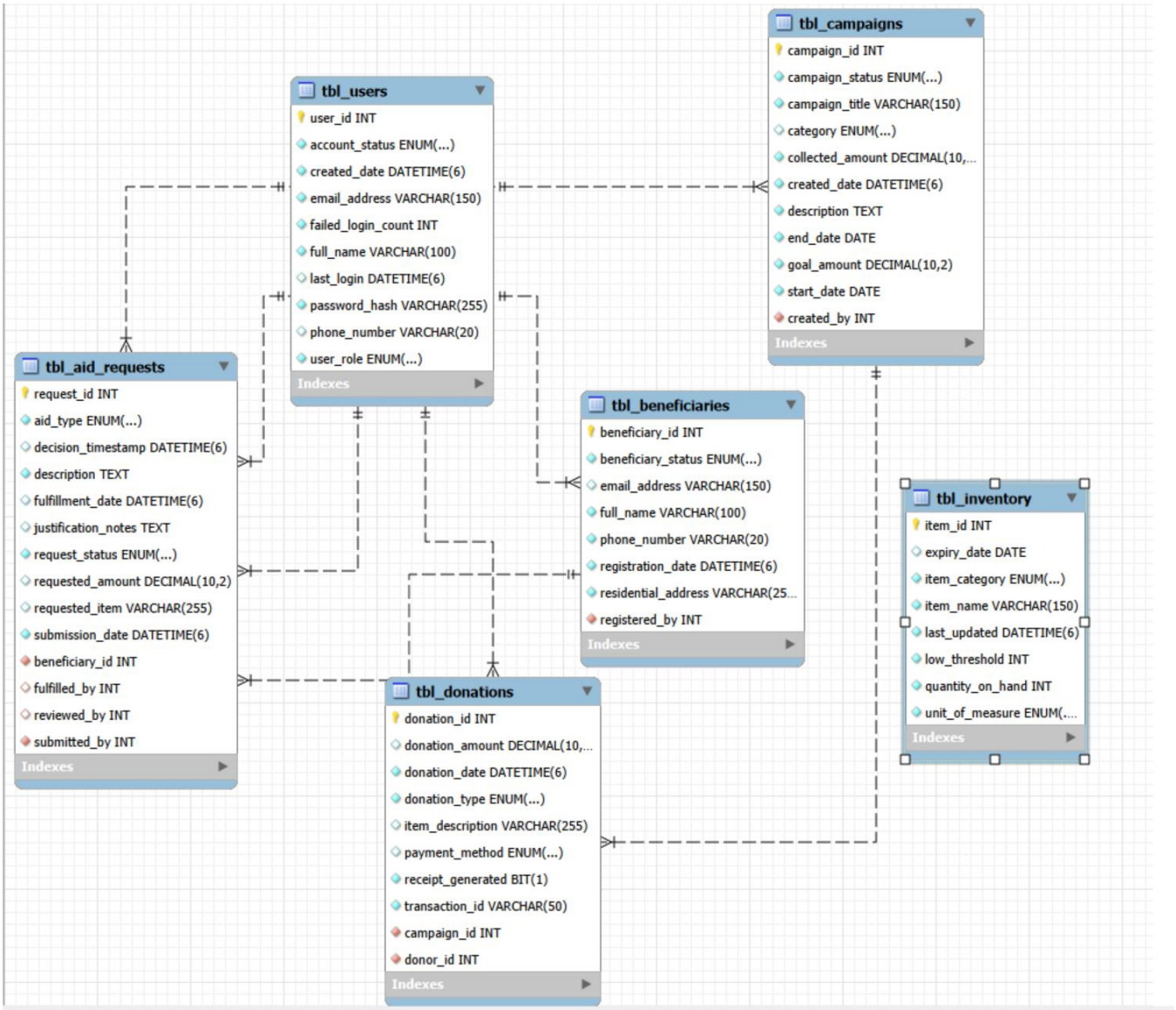
ID	Field Name (DB)	Data Type	Width	Required	Valid Values / Default	Description
PK	**REQUEST_ID**	INT	—	Yes	Auto-increment	Unique identifier for each aid request.
FK	**BENEFICIARY_ID**	INT	—	Yes	References tbl_beneficiaries.BENEFICIARY_ID	Beneficiary the request is filed on behalf of.
	AID_TYPE	VARCHAR	30	Yes	Dropdown: Monetary | Food | Supplies | Medical | Other	Category of aid being requested.
	DESCRIPTION	TEXT	1000	Yes	Free text	Detailed description and justification of the aid needed.
	REQUESTED_AMOUNT	DECIMAL	10,2	Cond.	Required if AID_TYPE = Monetary; must be > 0	Monetary amount

ID	Field Name (DB)	Data Type	Width	Required	Valid Values / Default	Description
						requested in USD.
	REQUESTED_ITEM	VARCHAR	255	Cond.	Required if AID_TYPE is non-monetary	Description of in-kind items requested.
	REQUEST_STATUS	VARCHAR	20	Yes	Dropdown: Pending | Approved | Denied | Fulfilled. Default: Pending	Current processing status of the aid request.
	JUSTIFICATION_NOTES	TEXT	1000	Cond.	Required on Denial; recommended on Approval	Case manager notes documenting the basis for the decision.
	DECISION_TIMESTAMP	DATETIME	—	No	System-generated on decision	Timestamp when the approval or denial was recorded.
FK	**REVIEWED_BY**	INT	—	No	References tbl_users.USER_ID	User ID of the case manager who reviewed the request.
FK	**SUBMITTED_BY**	INT	—	Yes	References tbl_users.USER_ID	User ID of the staff member who submitted the request.
	SUBMISSION_DATE	DATETIME	—	Yes	System-generated	Timestamp of initial submission.
	FULFILLMENT_DATE	DATETIME	—	No	System-generated on fulfillment	Timestamp when aid delivery was confirmed.
FK	**FULFILLED_BY**	INT	—	No	References tbl_users.USER_ID	User ID of the staff member who recorded fulfillment.

7.6 tbl_inventory

Stores in-kind inventory items donated to the organization. QUANTITY_ON_HAND is decremented atomically within a @Transactional operation when an in-kind aid request is fulfilled. If QUANTITY_ON_HAND falls below LOW_THRESHOLD after a transaction, the system triggers a staff alert.

ID	Field Name (DB)	Data Type	Width	Required	Valid Values / Default	Description
PK	**ITEM_ID**	INT	—	Yes	Auto-increment	Unique identifier for each in-kind inventory item.
	ITEM_NAME	VARCHAR	150	Yes	Free text	Descriptive name of the item.
	ITEM_CATEGORY	VARCHAR	50	Yes	Dropdown: Food | Clothing | Medical | Supplies | Other	Classification of the inventory item.
	QUANTITY_ON_HAND	INT	—	Yes	Must be >= 0	Current available stock quantity.
	LOW_THRESHOLD	INT	—	Yes	Must be > 0	Quantity level that triggers a low-stock system alert.
	UNIT_OF_MEASURE	VARCHAR	30	Yes	Dropdown: Units | Kg | Liters | Boxes | Bags	Unit used to measure the item quantity.
	EXPIRY_DATE	DATE	—	No	Optional; for perishables	Expiry date for time-sensitive items.
	LAST_UPDATED	DATETIME	—	Yes	System-generated on change	Timestamp of last quantity update.



7.7 Table Relationships Summary

The following table summarizes all foreign key relationships between tables, the cardinality of each relationship, and the referential integrity action applied.

Parent Table (PK)	Child Table (FK)	Cardinality	Description
tbl_users (USER_ID)	tbl_donations (DONOR_ID)	One-to-Many (1:N)	One donor user can make many donation transactions.
tbl_campaigns (CAMPAIGN_ID)	tbl_donations (CAMPAIGN_ID)	One-to-Many (1:N)	One campaign can receive many donations.
tbl_users (USER_ID)	tbl_campaigns (CREATED_BY)	One-to-Many (1:N)	One staff/admin user can create many campaigns.
tbl_beneficiaries (BENEFICIARY_ID)	tbl_aid_requests (BENEFICIARY_ID)	One-to-Many (1:N)	One beneficiary can have many aid requests over time.

Parent Table (PK)	Child Table (FK)	Cardinality	Description
tbl_users (USER_ID)	tbl_aid_requests (SUBMITTED_BY)	One-to-Many (1:N)	One staff member can submit many aid requests.
tbl_users (USER_ID)	tbl_aid_requests (REVIEWED_BY)	One-to-Many (1:N)	One case manager can review many aid requests.
tbl_users (USER_ID)	tbl_aid_requests (FULFILLED_BY)	One-to-Many (1:N)	One staff member can fulfill many aid requests.
tbl_users (USER_ID)	tbl_beneficiaries (REGISTERED_BY)	One-to-Many (1:N)	One staff member can register many beneficiaries.

8. Report Definitions

This section defines the technical specification for each report and dashboard view generated by the Charity and Aid Management System. Reports are rendered server-side by the Spring Boot reporting service layer and delivered to the client as JSON (for on-screen display via the HTML/JS frontend), as PDF (for donation receipts), or as CSV (for export). All reports enforce role-based access at the API level. Date range filtering is available on all reports as a minimum; additional filter dimensions are specified per report below.

ID	Report Name	Format	Roles with Access	Fields / Dimensions Included
RPT-01	Donation Summary	Screen / CSV	Admin, Staff	Date range, Donor name, Campaign title, Donation type, Amount, Transaction ID, Payment method. Totals by campaign and period.
RPT-02	Active Campaigns	Screen / PDF	Admin, Staff, Leadership	Campaign title, Category, Start date, End date, Goal amount, Collected amount, Progress %, Status, Created by.
RPT-03	Pending Aid Requests	Screen	Case Manager, Admin	Request ID, Beneficiary name, Aid type, Requested amount/item, Submission date, Days pending, Submitted by.
RPT-04	Fulfilled Aid Report	Screen / PDF	Admin, Staff, Leadership	Request ID, Beneficiary, Aid type, Approval date, Fulfillment date, Fulfilled by, Resources used.
RPT-05	Campaign Performance	Screen / PDF	Admin, Leadership	Campaign title, Donations over time (trend), Number of donors, Goal vs collected, Percentage achieved.
RPT-06	Aid Distribution Summary	Screen / PDF	Admin, Leadership	Aid type breakdown (monetary, food, supplies, medical), Number of beneficiaries served, Total value distributed, Period.
RPT-07	Approval Turnaround	Screen	Admin	Average hours from submission to decision, by Case Manager, by Aid type, over selected period.
RPT-08	Compliance / Audit	Screen / PDF	Admin only	User, Action type, Entity affected, Timestamp, IP address. Covers all CRUD events, logins, approvals, denials.
RPT-09	Inventory Status	Screen	Staff, Admin	Item name, Category, Quantity on hand, Low threshold, Unit, Expiry date. Items below threshold highlighted in amber.
RPT-10	Leadership KPI Dashboard	Screen (real-time)	Leadership, Admin	Total donations to date, Active campaigns count, Open aid requests count, Fulfilled this month, Donation trend chart (12 months), Aid distribution donut chart, Resource availability table.

9. API and Integration Design

All client-server communication in the Charity and Aid Management System is performed through a RESTful API built with Spring MVC (@RestController). All endpoints return JSON responses and consume JSON request bodies. Every endpoint is secured by Spring Security — unauthenticated requests receive HTTP 401, insufficient role receives HTTP 403. The base URL for all endpoints is https://{deployment-host}/api. Authentication endpoints (/auth/*) are public. All other endpoints require a valid JWT in the Authorization: Bearer header.

Standard HTTP response codes are used throughout: 200 OK for successful reads, 201 Created for successful resource creation, 400 Bad Request for validation failures, 401 Unauthorized for missing/invalid tokens, 403 Forbidden for insufficient role, 404 Not Found for missing resources, and 500 Internal Server Error for unhandled exceptions. All error responses include a JSON body with a code, message, and timestamp field.

ID	Method	Endpoint	Auth / Role	Description
API-01	POST	/api/auth/register	Public	Register a new donor account with name, email, and password.
API-02	POST	/api/auth/login	Public	Authenticate user credentials and return a signed JWT token.
API-03	POST	/api/auth/logout	Authenticated	Invalidate the current JWT session token.
API-04	POST	/api/auth/reset-password	Public	Initiate a secure password reset via email verification link.
API-05	GET	/api/users	Admin	Retrieve paginated list of all system user accounts.
API-06	GET	/api/users/{id}	Admin, Self	Retrieve details of a specific user account by ID.
API-07	PUT	/api/users/{id}/role	Admin	Update the role assigned to a user account.
API-08	PUT	/api/users/{id}/status	Admin	Change account status: Active, Inactive, or Suspended.
API-09	GET	/api/campaigns	Authenticated	Retrieve paginated list of all campaigns with status and progress.
API-10	POST	/api/campaigns	Staff, Admin	Create a new fundraising campaign.
API-11	PUT	/api/campaigns/{id}	Staff, Admin	Update the details of an existing campaign.
API-12	PUT	/api/campaigns/{id}/close	Staff, Admin	Manually close an active campaign.
API-13	DELETE	/api/campaigns/{id}	Admin	Delete a campaign with zero donations.
API-14	POST	/api/donations	Donor	Record a new monetary or in-kind donation.
API-15	GET	/api/donations	Admin, Staff	Retrieve all donation records with optional filters.

ID	Method	Endpoint	Auth / Role	Description
API-16	GET	/api/donations/my	Donor	Retrieve the authenticated donor's personal donation history.
API-17	GET	/api/donations/{id}/receipt	Donor, Admin	Generate and return the PDF receipt for a specific donation.
API-18	POST	/api/beneficiaries	Staff, Admin	Register a new beneficiary.
API-19	GET	/api/beneficiaries	Staff, Admin, CM	Retrieve all registered beneficiaries with search support.
API-20	PUT	/api/beneficiaries/{id}	Staff, Admin	Update a beneficiary's profile information.
API-21	PUT	/api/beneficiaries/{id}/status	Staff, Admin	Deactivate or reactivate a beneficiary record.
API-22	POST	/api/aid-requests	Staff	Record a new aid request for a registered beneficiary.
API-23	GET	/api/aid-requests	Staff, CM, Admin	Retrieve all aid requests with optional status filter.
API-24	GET	/api/aid-requests/{id}	Staff, CM, Admin	Retrieve full details of a specific aid request.
API-25	PUT	/api/aid-requests/{id}/approve	Case Manager, Admin	Approve a pending aid request with justification notes.
API-26	PUT	/api/aid-requests/{id}/deny	Case Manager, Admin	Deny a pending aid request with mandatory justification notes.
API-27	PUT	/api/aid-requests/{id}/fulfill	Staff	Record fulfillment of an approved aid request.
API-28	GET	/api/inventory	Staff, Admin	Retrieve all inventory items with quantities and threshold status.
API-29	POST	/api/inventory	Staff, Admin	Create a new inventory item record.
API-30	PUT	/api/inventory/{id}	Staff, Admin	Update quantity or details of an inventory item.
API-31	GET	/api/reports/donations	Admin, Staff	Generate donation summary report with date and campaign filters.
API-32	GET	/api/reports/campaigns	Admin, Staff, Leadership	Generate campaign performance report.
API-33	GET	/api/reports/aid	Admin, Leadership	Generate aid distribution summary report.
API-34	GET	/api/reports/audit	Admin	Generate compliance and audit log report.

ID	Method	Endpoint	Auth / Role	Description
API-35	GET	/api/dashboard/kpi	Admin, Leadership	Retrieve real-time KPI metrics for the leadership dashboard.

10. Sprint Development Plan

Development is organized into three two-week sprints following an agile iterative approach. Each sprint produces a working, deployable version of the system that is reviewed by the advisor at the end of the sprint period. The sprint plan below defines the key deliverables, date range, estimated hours, and current status for each sprint. All sprints are preceded by environment setup (Sprint 0) and followed by final documentation and demonstration activities.

Sprint	Key Deliverables	Dates	Hours	Status
Sprint 1	Set up development environment; configure Spring Boot project, MySQL schema, Spring Security JWT auth; implement User Management module (registration, login, RBAC); deploy skeleton to Railway/Render.	03/02/2026 – 03/29/2026	38	In Progress
Sprint 2	Implement Campaign Management, Donation Recording (with PDF receipt generation), Beneficiary Management, and Aid Request submission workflow; frontend HTML5/Bootstrap pages for all Sprint 2 features.	03/30/2026 – 04/12/2026	30	Planned
Sprint 3	Implement Case Manager review workflow (approve/deny/escalate), Aid Fulfillment with inventory deduction, Reporting and KPI Dashboard, Notification service (SendGrid SMTP), full system testing and bug fixes.	04/13/2026 – 05/01/2026	30	Planned

10.1 Sprint 1 — Foundation and Authentication

Sprint 1 establishes the technical foundation of the entire system. The development environment is configured with Java 17, Spring Boot 3, Maven, MySQL 8, and IntelliJ IDEA. The GitHub repository is created with the project structure, Maven dependencies, and GitHub Actions CI workflow. The MySQL schema is created for all six tables with correct data types, primary keys, foreign keys, and constraints. The Spring Boot application is configured with Spring Security JWT authentication, BCrypt password encoding, and @PreAuthorize role enforcement. The User Management API (FR-01 through FR-16) is fully implemented and tested. A working login, registration, and role-assignment workflow is demonstrated at the Sprint 1 advisor review on March 29, 2026.

10.2 Sprint 2 — Core Business Modules

Sprint 2 implements the four primary operational modules: Campaign Management (FR-30 to FR-40), Donation Recording (FR-17 to FR-29, FR-34 receipt generation), Beneficiary Management (FR-41 to FR-46), and Aid Request Submission (FR-47 to FR-58). The SendGrid SMTP integration is completed for donation confirmation emails. All Bootstrap 5 frontend pages for Sprint 2 features are implemented and connected to the live API. The system is deployed to Railway and accessible at a public HTTPS URL for the Sprint 2 review on April 12, 2026.

10.3 Sprint 3 — Workflow, Reporting, and Testing

Sprint 3 completes the system with the Case Manager review workflow (FR-52 to FR-64 approve/deny/escalate), Aid Fulfillment with inventory deduction (FR-65 to FR-70), all Reporting and Dashboard endpoints (FR-77 to FR-90), Notification enhancements (FR-91 to FR-94), and the Audit trail (FR-88 to FR-90). Unit tests are written using JUnit 5 and Mockito for all service layer methods. Integration tests are written for all API endpoints using Spring MockMvc. A simulated User Acceptance Testing session is conducted using the five role accounts. All identified defects are resolved before the Sprint 3 advisor review on May 2, 2026.

11. Client Approval

The following individuals have reviewed this Design Documentation and approve it as the authoritative technical design baseline for the Charity and Aid Management System. No significant design changes may be made without the written consent of the approving parties and a formal revision of this document.

Name	Role	Signature / Date
Shivani Pothkanuri	Author / Student Developer	
Jeffrey Tirschman	Advisor / Instructor	
Graduate Program Director	Graduate Program Director	